Amplifier × Workgraph Integration Proposal

Date: 2026-02-18 **Author**: analyst agent (synthesized from three research documents) **Status**: Draft for decision

---

1. Executive Summary

Integrating Amplifier with workgraph would let wg dispatch tasks to full Amplifier sessions — giving each task access to Amplifier's multi-agent delegation, bundle ecosystem, and provider abstraction — while Amplifier sessions could decompose complex work into wg graphs for parallel execution. The research shows that most of this integration already exists as a thin adapter bundle (`amplifier-bundle-workgraph`), and the real question is not "can we integrate" but "is it worth formalizing." The answer depends on whether you expect to use Amplifier as a primary execution environment. If you do, the executor-side changes are small (5 files, 200 lines) and unlock a clean plugin story for any future executor, not just Amplifier. If you don't, the bundle already works with a minor hack and no changes to wg are needed.

---

2. Option A: Amplifier as a wg Executor

What this means: `wg service start` spawns Amplifier sessions instead of (or alongside) Claude CLI sessions. Each task gets a full Amplifier environment with bundles, tools, and multi-agent delegation.

What changes

wg currently hardcodes stdin piping to `type = "claude"` executors only. All other types get `Stdio::null()`, silently discarding the prompt. The amplifier bundle works around this by declaring `type = "claude"` and using a wrapper script that bridges stdin to a positional argument. Formalizing this requires:

1. **Add `prompt_mode` to `ExecutorSettings`** — `"stdin"` (default, current behavior), `"file"` (write prompt.txt, don't pipe), `"arg"` (pass as CLI argument), `"none"`. About 40 lines in `spawn.rs` + `executor.rs`.
2. **Add `{{model}}` template variable** — so executor configs can put the model anywhere in their args. 20 lines in `executor.rs`.
3. **Always write `prompt.txt`** regardless of executor type. Currently only written for `type = "claude"`. Trivial change.
4. **Optionally**: add a built-in `"amplifier"` default config alongside `"claude"`, `"shell"`, and `"default"` in `executor.rs:315-418`.

What's easy

- The template variable system (`{{task_id}}`, `{{task_context}}`, etc.) already works for any executor. No changes needed.
- The `run.sh` wrapper pattern (auto-mark done/failed on exit) is executor-agnostic. Works today.
- Environment variable passing (`WG_TASK_ID`, `WG_AGENT_ID`) works for all types.
- Executor TOML files already override built-in defaults cleanly.

What's hard

- **Nothing is architecturally hard.** The changes are small and well-scoped. The gap analysis identifies exactly 6 lines of branching logic in `spawn.rs:221-261` that need refactoring.
- The only complexity is deciding what `prompt_mode` values to support and making the choice backward-compatible (existing `type = "claude"` configs must keep working).

Effort estimate

200 lines of code across `spawn.rs`, `executor.rs`, and `config.rs`. Touches well-understood code paths with existing test coverage. A competent PR.

---

3. Option B: wg as an Amplifier Bundle (Status Quo)

What this means: Amplifier sessions detect complex tasks and delegate to wg. This is what `amplifier-bundle-workgraph` already does. The bundle installs a behavior (`workgraph.yaml`), a planner agent (`workgraph-planner.md`), and context documents (`workgraph-guide.md`) that teach Amplifier agents how to decompose work into wg graphs.

Is this sufficient?

For the use case of "I'm running Amplifier and encounter a complex task," yes. The bundle:

- Detects when tasks have 4+ subtasks with parallelism opportunities
- Delegates to a planner agent that knows fan-out/fan-in, pipeline, and iterative patterns
- Builds a wg graph with `wg init` / `wg add` / `wg service start`
- The wg daemon dispatches tasks using whatever executor is configured (currently claude)

What's missing

1. **Circular delegation is blocked.** If wg's executor is claude (not amplifier), the spawned agents don't have Amplifier's bundles or multi-agent capabilities. You get wg decomposition → claude execution, but not wg decomposition → amplifier execution. This requires Option A to close the loop.
2. **No way for wg-spawned agents to use Amplifier bundles.** Even if an agent needs a specialized Amplifier bundle for its task, it runs as bare claude. This limits the value of Amplifier's ecosystem when wg is coordinating.
3. **The bundle is maintained externally** (`ramparte/amplifier-bundle-workgraph`). If wg's executor protocol or template variables change, the bundle breaks silently.

Verdict

Option B is sufficient if you use Amplifier purely as a top-level entry point and don't need Amplifier capabilities in wg-spawned task agents. It's not sufficient if you want wg-spawned agents to access Amplifier's bundle ecosystem.

---

4. Option C: Full Bidirectional Integration

What this means: Both directions work simultaneously. Amplifier sessions can delegate to wg (bundle), and wg can spawn Amplifier sessions (executor). The architecture becomes recursive — an Amplifier agent decomposes work into a wg graph, wg spawns Amplifier sessions for each task, and those sessions could theoretically decompose further.

Is the complexity worth it?

The incremental complexity over Option A is near zero. Once wg can spawn Amplifier sessions (Option A), bidirectional integration is just Option A + Option B running together. No new code is needed — the bundle already works, and adding the executor changes from Option A completes the other direction.

The only real concern is **recursion depth control**: an Amplifier agent that decomposes into wg tasks that spawn Amplifier agents that decompose further. This is self-limiting in practice (each level has context window costs and latency), but it should be called out. A simple mitigation: pass a `WG_DEPTH` environment variable and have the planner skip decomposition beyond depth 2.

Verdict
If you do Option A at all, you get Option C for free. The question isn't "A or C" — it's "A or nothing."

---

5. The Bundle Question

Why does Amplifier use bundles?
Bundles are Amplifier's distribution and composition unit. They package behaviors (what an agent does), context (what it knows), and agents (specialized sub-agents it can delegate to) into a single installable unit referenced by namespace (`workgraph:workgraph-planner`). They're distributed via git URLs and composed via `includes`.

Bundles solve a real problem: how do you share agent configurations across teams and projects? The `amplifier bundle add` workflow is clean — point at a repo, get a packaged set of capabilities.

Should wg adopt this for executor configs?
No. wg's executor TOML files are already simpler and sufficient.

Consider what wg needs to package per executor: - A TOML config (command, args, env, prompt template) - Optionally a wrapper script

This is 1-2 files. wg's current model — drop files in `.workgraph/executors/` — is the right level of abstraction. Bundles add namespace resolution, version management, include graphs, and a registry — machinery that makes sense when you have dozens of composable behaviors, but is over-engineering for a few executor configs.

Where bundles become interesting is if wg wanted to package **entire project templates** (executor config + agency roles + task templates + skills). But that's a different feature from executor packaging, and simpler solutions exist (e.g., `wg init --template <git-url>` that clones a `.workgraph/` skeleton).

Verdict
Don't adopt the bundle model for executor configs. If project templating becomes a need, solve it directly with `wg init --template` rather than building a generic package system.

---

6. Concrete Next Steps

Ordered by effort (ascending) and value (descending).

PR 1: Always write `prompt.txt` for all executor types
Effort: Trivial (< 10 lines) **Value**: Debugging — every spawned task gets a readable prompt file regardless of executor type. Currently only written for `type = "claude"`. **Files**: `src/commands/spawn.rs`

PR 2: Add `{{model}}` template variable
Effort: Small (30 lines) **Value**: Unlocks model selection for any executor via args, not just the hardcoded `--model` flag for claude. **Files**: `src/service/executor.rs` (TemplateVars struct + apply method)

PR 3: Add `prompt_mode` to decouple stdin piping from executor type
Effort: Medium (100 lines) **Value**: This is the core change. Eliminates the `type = "claude"` hack. Enables any executor to receive prompts via stdin, file, CLI arg, or not at all. Backward-compatible: `type = "claude"` defaults to `prompt_mode = "stdin"`. **Files**: `src/commands/spawn.rs`, `src/service/executor.rs`

PR 4: Enrich `build_task_context()` with dependency metadata
Effort: Small (40 lines) **Value**: Include dependency titles in `{{task_context}}` so downstream agents know what each upstream task was, not just what files it produced. Benefits all executors. **Files**: `src/commands/spawn.rs`

PR 5: Include `verify` field in default prompt template
Effort: Trivial (5 lines) **Value**: Agents learn what "done" means. Currently the `verify` field exists but isn't passed to the agent. **Files**: `src/service/executor.rs` (default prompt template)

PR 6 (optional): Built-in amplifier executor default
Effort: Small (50 lines) **Value**: `wg config coordinator.executor amplifier` works out of the box without installing any files. Nice-to-have once PR 3 lands. **Files**: `src/service/executor.rs` (default_config function)

Not recommended now
- **Artifact content inlining** (R5 from context transfer analysis): High effort, edge cases around binary files and size limits. Wait until it's a felt need.
- **Transitive context**: Passing context from grandparent dependencies adds context bloat. The current "re-export via artifacts" pattern is deliberate.
- **Per-task executor selection**: Useful but orthogonal to amplifier integration. Separate initiative.

---

7. Risks and Concerns

If we integrate
1. **Maintenance coupling.** Adding amplifier as a first-class executor means tracking Amplifier's CLI changes (flag names, output format, session semantics). Amplifier is a Microsoft project; its stability guarantees are unknown.
2. **Testing complexity.** E2E tests that spawn Amplifier sessions require Amplifier to be installed and configured. CI either needs Amplifier or the tests need mocking. The bundle repo's test suite already shows 120-second timeouts for lifecycle tests.
3. **Recursion risk.** Bidirectional integration creates the possibility of unbounded recursion (amplifier → wg → amplifier → ...). Mitigated by depth limits, but needs explicit handling.
4. **User confusion.** Two orchestrators (Amplifier and wg's service daemon) both managing agents is conceptually complex. Clear documentation is essential to explain when each layer is active.

If we don't integrate
1. **The `type = "claude"` hack persists.** Every non-claude executor will use `type = "claude"` as a workaround, eroding the meaning of the type field. This is already happening with the amplifier bundle.
2. **Missed generalization.** PRs 1-3 benefit all future executors, not just Amplifier. Delaying them means every new executor hits the same stdin piping wall.
3. **The amplifier bundle remains a standalone workaround.** It works, but it's fragile — tightly coupled to wg's undocumented internal behavior (which executor types get stdin piping).

Recommendation
Do PRs 1-5. They're small, backward-compatible, and improve wg's executor model for everyone — not just Amplifier. Total effort: 200 lines across 3 files. Skip PR 6 unless you actively use Amplifier and want the convenience.

The amplifier bundle already works. The real win here is not "amplifier integration" — it's fixing wg's executor model so that it cleanly supports any agent runtime, with Amplifier as the motivating example.